Reviewer Pack — Minimal Symplectic Leaf Count and Communication Complexity R…

Entry be3fb124156c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 01:26:00 UTC

Minimal Symplectic Leaf Count and Communication Complexity Rank Variance Ratio

Entry ID: `be3fb124156c` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 01:26:00 UTC

1. Conjecture

Field A (mathematical branch): Symplectic Geometry **Field B** (complexity object): Communication Complexity

Statement:

> For any given Boolean function f with n variables, the communication complexity rank variance ratio (CRVR(f)) of its associated communication complexity is upper-bounded by the minimal number of symplectic leaves in the symplectic leaf decomposition of the moment map associated with the dual of f's characteristic polynomial, i.e., CRVR(f) ≤ Θ(min_symplectic_leaves(n)).

Rationale (proposer's reasoning):

> Symplectic geometry offers a structured way to analyze complex systems through phase spaces and their decompositions. The conjecture suggests that the structure in symplectic leaves might reflect the complexity of communication protocols, potentially leading to new insights into the hardness of computation.

Taxonomy category: `symplectic_geometry_communication_complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `981ea0cb55829c22`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the mean communication complexity rank variance ratio (CRVR) across all Boolean functions is less than or equal to the mean minimal symplectic leaf count, with no seed having a CRVR greater than this mean by more than 3 standard deviations.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "communication complexity rank variance ratio" AND "symplectic geometry" AND "moment map" - "characteristic polynomial dual" AND "symplectic leaf count" AND "communication complexity" - "Boolean function" AND "CRVR(f)" AND "minimal symplectic leaves"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.4s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot row
        max_row = i
        for k in range(i+1, n):
            if abs(A[k][i]) > abs(A[max_row][i]):
                max_row = k
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate non-pivot elements
        factor = Fraction(-A[i][i], A[i][i])
        for j in range(i, n):
            A[i][j] /= A[i][i]
        for k in range(n):
            if k != i:
                factor = Fraction(A[k][i], A[i][i])
                for j in range(i, n):
                    A[k][j] += factor * A[i][j]
    return A

def determinant(A):
    n = len(A)
    det = 1
    for i in range(n):
```

```
        if A[i][i] == 0:
            return 0
        det *= A[i][i]
    return det

def communication_complexity_rank_variance_ratio(f):
    # Placeholder implementation
    n = len(f)
    A = [[Fraction(1, 2) if i != j else Fraction(-1, 2) for j in range(n)] for i in range(n)]
    U = gaussian_elimination(A)
    rank = sum(1 for row in U if any(row[j] != 0 for j in range(n)))
    return rank / n

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    f = [random.choice([0, 1]) for _ in range(2**n)]

    crvr = communication_complexity_rank_variance_ratio(f)
    min_symplectic_leaves = n  # Placeholder value

    return {
        "metric_name": "CRVR",
        "metric_value": crvr,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": crvr <= min_symplectic_leaves,
        "counterexample": "" if crvr <= min_symplectic_leaves else f"CRVR={crvr} > min_symplectic_
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 6)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_crvr = sum(r["metric_value"] for r in results) / len(results)
    std_crvr = math.sqrt(sum((r["metric_value"] - mean_crvr)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_crvr} std={std_crvr} support_fraction={support_fracti
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"CRVR > min_symplectic_leaves\" first_failing_se
    else:
        print("RESULT: INCONCLUSIVE insufficient evidence")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, which prevents us from evaluating whether the conjecture is supported or falsified. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13552
2	propose	ollama_remote	glm4:latest	0	0	13432
3	preregistration	ollama_remote	glm4:latest	0	0	10173
4	novelty	ollama_remote	glm4:latest	0	0	8538
5	novelty	ollama_remote	glm4:latest	0	0	9022
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	55485
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11467
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	39248
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12358
10	judge	ollama_remote	glm4:latest	0	0	66412

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 239687 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/be3fb124156c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/be3fb124156c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/be3fb124156c.tar.gz` (if generated)